

# Содержание

|                                                        |    |
|--------------------------------------------------------|----|
| 1. Введение .....                                      | 2  |
| 2. Зависимости .....                                   | 3  |
| 3.1. Интеграция с CMake .....                          | 4  |
| 3.2. Интеграция с QMake .....                          | 7  |
| 4. Класс Config .....                                  | 8  |
| 4.1. Формат JSON .....                                 | 23 |
| 4.2. Формат INI .....                                  | 25 |
| 4.3. Формат XML .....                                  | 27 |
| 4.4. Формат YAML .....                                 | 28 |
| 4.5. Комментирование значений внутри конфига .....     | 29 |
| 5. Сетевое взаимодействие .....                        | 32 |
| 5.1. Сокеты .....                                      | 33 |
| 5.2. Поток для автоматизации работы сокетов .....      | 34 |
| 5.3. Сообщения (класс Message) .....                   | 35 |
| 5.4. Функции-утилиты для сетевого взаимодействия ..... | 36 |
| 6. Дополнительные функции-утилиты .....                | 37 |

# 1. Введение

Библиотека SimpleAPI задумывалась как инструмент для удобной работы с файлами конфигураций с возможностью использования комментариев внутри файлов. А также передача конфигов по сети в указанном формате(**TODO**) (без комментариев) между программами.

То есть подразумевается несколько сценариев взаимодействия с конфигами:

- программа-человек: файлы с возможностью комментирования
- программа-программа: простая, с точки зрения программиста, упаковка значений в строку конфига, её передача до другого компьютера через сокет и простая распаковка в соответствующие значения на другом компьютере

## Реализовано:

- полная поддержка формата JSON
- поддержка формата INI (свободно конвертируется в JSON и обратно)
- возможность указать стиль комментариев (символы границы 1-2 знака для однострочных и 1-3 знака - для многострочных), их чтение из файла и запись в него
- сетевое взаимодействие сервер-клиент в формате UDP с автоматической фрагментацией и дефрагментацией пакетов

## В планах:

- поддержка XML
- поддержка YAML
- шифрование при отправке по сети

## 2. Зависимости

### Основные требования к системе

ОС: Linux

Стандарт языка: C++11

### Библиотеки

Проект зависит только от библиотеки pthread для класса SocketThread.

## 3.1. Интеграция с CMake

Минимальная версия CMake для сборки библиотеки 3.5.

**Библиотеку можно добавить несколькими способами:**

### Как самостоятельный CMake проект

```
cd <SimpleAPI_dir>
mkdir -p build
cd build
cmake [-DCMAKE_BUILD_TYPE=Debug] .. # флаг -DCMAKE_BUILD_TYPE указывается
опционально
make -j$(nproc)
```

В директории build появится директория lib с собранной библиотекой в формате .so и директорией include со всеми заголовочными файлами библиотеки.

### Как часть другого CMake проекта (add\_subdirectory)

В составе вызывающего CMakeLists.txt следует указать следующий код CMake:

```
set(SIMPLE_API_MAIN_DIR ${CMAKE_CURRENT_LIST_DIR}/../..) # указать путь до
базовой директории библиотеки
include(${SIMPLE_API_MAIN_DIR}/utils.cmake) # для доступа к макросам библиотеки
...
# код для определения типа сборки
if(CMAKE_BUILD_TYPE STREQUAL "Release")
    set(SimpleAPI_lib_name SimpleAPI)
else()
    set(SimpleAPI_lib_name SimpleAPIId)
endif()

add_subdirectory(${SIMPLE_API_MAIN_DIR}/SimpleAPI
${CMAKE_CURRENT_BINARY_DIR}/${SimpleAPI_lib_name})
```

```
# код для создания таргета основного приложения
add_executable(...)
...

# предполагается, что имя таргета совпадает с именем проекта (PROJECT_NAME)
# добавление зависимости от библиотеки для созданного таргета
target_link_libraries(${PROJECT_NAME} PUBLIC ${SimpleAPI_lib_name})
target_include_directories(${PROJECT_NAME} PUBLIC
${SIMPLE_API_MAIN_DIR}/SimpleAPI)
add_dependencies(${PROJECT_NAME} ${SimpleAPI_lib_name})
```

## Через использование технологии find\_package

Для использования этого варианта должна быть установлена библиотека `pkg-config`:

```
sudo apt-get install pkg-config # Ubuntu
```

Пример кода CMake

```
set(SIMPLE_API_MAIN_DIR ${CMAKE_CURRENT_LIST_DIR}/../..) # указать путь до
базовой директории библиотеки
include(${SIMPLE_API_MAIN_DIR}/utils.cmake) # для доступа к макросам библиотеки

# код для создания таргета основного приложения
add_executable(...)
...

# предполагается, что имя таргета совпадает с именем проекта (PROJECT_NAME)
if(CMAKE_BUILD_TYPE STREQUAL "Release")
    find_package(SimpleAPI REQUIRED)
    if(SimpleAPI_FOUND)
        target_link_libraries(${PROJECT_NAME} PUBLIC SimpleAPI)
        target_include_directories(${PROJECT_NAME} PUBLIC
${SimpleAPI_INCLUDE_DIRS})
    else()
        message(FATAL "SimpleAPI not found")
    endif()
endif()
```

```
endif(SimpleAPI_FOUND)
else()
  find_package(SimpleAPI REQUIRED)
  if(SimpleAPI_FOUND)
    target_link_libraries(${PROJECT_NAME} PUBLIC SimpleAPI)
    target_include_directories(${PROJECT_NAME} PUBLIC
${SimpleAPI_INCLUDE_DIRS})
  else()
    message(FATAL "SimpleAPI not found")
  endif(SimpleAPI_FOUND)
endif()
```

## 3.2. Интеграция с QMake

Ввиду ограниченности системы сборки QMake, реализовано только самое базовое включение SimpleAPI. Проект SimpleAPI не будет собираться как самостоятельная библиотека, будь то статичная или динамическая.

Для добавления в ваш QMake-проект SimpleAPI следует придерживаться такого шаблона:

(теоретический project.pro)

```
QT = core

CONFIG += c++11 cmdline

include(SimpleAPI_library/SimpleAPI.pri) # добавление SimpleAPI как источник
исходников

SOURCES += \
    main.cpp
```

## 4. Класс Config

Используется как основной класс, с которым взаимодействует пользователь библиотеки.

Конструкторы Config позволяют создавать объекты Config на основе разрешённых выбранным стандартом значений (long double, bool, std::string, const char\*, Config(ValueType::eJson), Config(ValueType::eArray), **TODO: Config(ValueType::XML), TODO: Config(ValueType::YAML)**).

Используйте файл Config.h как источник информации о доступных методах. Обращайте внимание на отметки API\_ в конце объявлений методов

Следующие списки могут быть устаревшими!

### Методы работы с комментариями

```
Config& setComment(const Comment& content) noexcept;
Config& setComment(const std::string &content_before, const std::string
&content_after) noexcept;
Config& setPrefixComment(const std::string& content) noexcept;
Config& setSuffixComment(const std::string& content) noexcept;

Comment& getComment() noexcept;
Comment getComment() const noexcept;
std::string getPrefixComment() const noexcept;
std::string getSuffixComment() const noexcept;

Config& clearComment() noexcept;
Config& clearPrefixComment() noexcept;
Config& clearSuffixComment() noexcept;
Config& deleteComment() noexcept;
Config& deletePrefixComment() noexcept;
Config& deleteSuffixComment() noexcept;

CommentDesign& getCommentDesign() noexcept;
CommentDesign getCommentDesign() const noexcept;
```



```
Config& setCommentDesign(const CommentDesign &design) noexcept;  
Config& clearCommentDesign() noexcept;
```

## Методы работы с комментариями элементов внутри контейнеров

(имя функции пишется через знак подчёркивания \_)

```
Config& set_comment(const size_t& index, const Comment &content);  
Config& set_comment(const std::string& key, const Comment &content);  
  
Config& set_comment(const size_t& index, const std::string &content_before,  
const std::string &content_after);  
Config& set_comment(const std::string& key, const std::string &content_before,  
const std::string &content_after);  
  
Config& set_prefix_comment(const size_t& index, const std::string &content);  
Config& set_prefix_comment(const std::string& key, const std::string &content);  
  
Config& set_suffix_comment(const size_t& index, const std::string &content);  
Config& set_suffix_comment(const std::string& key, const std::string &content);  
  
Comment& get_comment(const size_t& index);  
Comment& get_comment(const std::string& key);  
  
Comment get_comment(const size_t& index) const;  
Comment get_comment(const std::string& key) const;  
  
std::string get_prefix_comment(const size_t& index) const;  
std::string get_prefix_comment(const std::string& key) const;  
  
std::string get_suffix_comment(const size_t& index) const;  
std::string get_suffix_comment(const std::string& key) const;  
  
Config& clear_comment(const size_t& index);  
Config& clear_comment(const std::string& key);
```

```

Config& clear_prefix_comment(const size_t& index);
Config& clear_prefix_comment(const std::string& key);

Config& clear_suffix_comment(const size_t& index);
Config& clear_suffix_comment(const std::string& key);

Config& delete_comment(const size_t& index);
Config& delete_comment(const std::string& key);

Config& delete_prefix_comment(const size_t& index);
Config& delete_prefix_comment(const std::string& key);

Config& delete_suffix_comment(const size_t& index);
Config& delete_suffix_comment(const std::string& key);

```

## Методы установки значения

```

Config& setValue() noexcept;
Config& setValue(std::nullptr_t) noexcept;
Config& setValue(const Config& other) noexcept;
Config& setValue(Config&& other) noexcept;
Config& setValue(const tools::IElement& other) noexcept;
Config& setValue(tools::IElement&& other) noexcept;
Config& setValue(const bool other) noexcept;

Config& setValue(const long double& other) noexcept;
Config& setValue(long double&& other) noexcept;
__ONLY_NUMBER_TYPES__(T)
Config& setValue(const T& other) noexcept;
__ONLY_NUMBER_TYPES__(T)
Config& setValue(T&& other) noexcept;

Config& setValue(const std::string& other) noexcept;
Config& setValue(std::string&& other) noexcept;
__ONLY_STRING_TYPES__(T)
Config& setValue(const T& other) noexcept;
__ONLY_STRING_TYPES__(T)

```

```

Config& setValue(T&& other) noexcept;

Config& setValue(const tools::ElementArray& other) noexcept;
Config& setValue(tools::ElementArray&& other) noexcept;
Config& setValue(const tools::ElementJson& other) noexcept;
Config& setValue(tools::ElementJson&& other) noexcept;

// вложенные контейнеры (используют insert_*() ниже, но возвращают этот же
базовый элемент)
Config& set(const std::string& key, const Config& value);
Config& set(const std::string& key, Config&& value);
Config& set(const size_t& index, const Config& value);
Config& set(const size_t& index, Config&& value);
Config& set(const size_t& index, const std::string& key, const Config& value);
Config& set(const size_t& index, const std::string& key, Config&& value);

```

## Методы получения реализации значений

```

bool& getBool();
bool getBool() const;
long double& getNumber();
long double getNumber() const;
std::string& getString();
std::string getString() const;
bool error() const noexcept;
std::string getError() const noexcept;

// вложенные контейнеры
// @complex_key - список индексов/ключей:  -> el[1]["k1"][2]
Config& get_front();
Config get_front() const;

Config& get_at(const size_t& index);
Config get_at(const size_t& index) const;
Config& get_at(const std::string& key);
Config get_at(const std::string& key) const;
Config& get_at(const std::vector<OnlySizetOrString>& complex_key);

```

```

Config get_at(const std::vector<OnlySizetOrString>& complex_key) const;
//фикс для вызова через {}
Config& get_at(const std::initializer_list<OnlySizetOrString>& complex_key);
Config get_at(const std::initializer_list<OnlySizetOrString>& complex_key)
const;

Config& get_back();
Config get_back() const;

bool& get_front_bool();
bool get_front_bool() const;
long double& get_front_number();
long double get_front_number() const;
std::string& get_front_string();
std::string get_front_string() const;

bool& get_bool_at(const size_t& index);
bool get_bool_at(const size_t& index) const;
bool& get_bool_at(const std::string& key);
bool get_bool_at(const std::string& key) const;
bool& get_bool_at(const std::vector<OnlySizetOrString>& complex_key);
bool get_bool_at(const std::vector<OnlySizetOrString>& complex_key) const;
//фикс для вызова через {}
bool& get_bool_at(const std::initializer_list<OnlySizetOrString>& complex_key);
bool get_bool_at(const std::initializer_list<OnlySizetOrString>& complex_key)
const;

long double& get_number_at(const size_t& index);
long double get_number_at(const size_t& index) const;
long double& get_number_at(const std::string& key);
long double get_number_at(const std::string& key) const;
long double& get_number_at(const std::vector<OnlySizetOrString>& complex_key);
long double get_number_at(const std::vector<OnlySizetOrString>& complex_key)
const;
//фикс для вызова через {}
long double& get_number_at(const std::initializer_list<OnlySizetOrString>&
complex_key);
long double get_number_at(const std::initializer_list<OnlySizetOrString>&

```

```

complex_key) const;

std::string& get_string_at(const size_t& index);
std::string get_string_at(const size_t& index) const;
std::string& get_string_at(const std::string& key);
std::string get_string_at(const std::string& key) const;
std::string& get_string_at(const std::vector<OnlySizetOrString>& complex_key);
std::string get_string_at(const std::vector<OnlySizetOrString>& complex_key)
const;
//фикс для вызова через {}
std::string& get_string_at(const std::initializer_list<OnlySizetOrString>&
complex_key);
std::string get_string_at(const std::initializer_list<OnlySizetOrString>&
complex_key) const;

bool& get_bool_back();
bool get_bool_back() const;
long double& get_number_back();
long double get_number_back() const;
std::string& get_string_back();
std::string get_string_back() const;

## Методы очистки значений
//здесь просто сброс комментариев, сброс до значения по умолчанию
Config& clear() noexcept;
//очистка контейнера, главные комментарии и CommentDesign не сбрасывается
Config& clearContainer();

```

## Методы добавления значений в контейнер

(выдаст exception если тип неподходящий)

```

Config& insert_front(const Config& other);
Config& insert_front(Config&& other);
Config& insert_front(const std::string& key, const Config& other);
Config& insert_front(const std::string& key, Config&& other);

```

```

Config& insert_at(const size_t& index, const Config& other);
Config& insert_at(const size_t& index, Config&& other);
Config& insert_at(const std::string& key, const Config& other);
Config& insert_at(const std::string& key, Config&& other);
Config& insert_at(const size_t& index, const std::string& key, const Config&
other);
Config& insert_at(const size_t& index, const std::string& key, Config&& other);

Config& insert_at(const shared_VElement::iterator iterator, const Config&
other);
Config& insert_at(const shared_VElement::iterator iterator, Config&& other);

Config& insert_at(const shared_VPairElement::iterator iterator, const
std::string& key, const Config& other);
Config& insert_at(const shared_VPairElement::iterator iterator, const
std::string& key, Config&& other);

Config& insert_back(const Config& other);
Config& insert_back(Config&& other);
Config& insert_back(const std::string& key, const Config& other);
Config& insert_back(const std::string& key, Config&& other);

//если путь не существовал - будут созданы все необходимые array и json, чтобы
путь был достижим
//если указанный путь уже занят, то элемент по пути должен быть преобразован в
array
Config& insert_back_force(const VString& keys, const Config& other) noexcept;
Config& insert_back_force(const VString& keys, Config&& other) noexcept;

Config& insert_before(const std::string& before_key, const std::string& key,
const Config& other);
Config& insert_before(const std::string& before_key, const std::string& key,
Config&& other);
Config& insert_after(const std::string& after_key, const std::string& key,
const Config& other);
Config& insert_after(const std::string& after_key, const std::string& key,
Config&& other);

```

```

Config& push_front(const Config& other);
Config& push_front(Config&& other);
Config& push_front(const std::string& key, const Config& other);
Config& push_front(const std::string& key, Config&& other);

Config& push_at(const size_t& index, const Config& other);
Config& push_at(const size_t& index, Config&& other);
Config& push_at(const std::string& key, const Config& other);
Config& push_at(const std::string& key, Config&& other);

Config& push_back(const Config& other);
Config& push_back(Config&& other);
Config& push_back(const std::string& key, const Config& other);
Config& push_back(const std::string& key, Config&& other);

Config& push_back_force(const VString& keys, const Config& other) noexcept;
Config& push_back_force(const VString& keys, Config&& other) noexcept;

Config& push_before(const std::string& before_key, const std::string& key,
const Config& other);
Config& push_before(const std::string& before_key, const std::string& key,
Config&& other);
Config& push_after(const std::string& after_key, const std::string& key, const
Config& other);
Config& push_after(const std::string& after_key, const std::string& key,
Config&& other);

//обход explicit
__ONLY_ALLOWED_TYPES__(T)
Config& push_front(const T& other);
__ONLY_ALLOWED_TYPES__(T)
Config& push_front(T&& other);
__ONLY_ALLOWED_TYPES__(T)
Config& push_front(const std::string& key, const T& other);
__ONLY_ALLOWED_TYPES__(T)
Config& push_front(const std::string& key, T&& other);
__ONLY_ALLOWED_TYPES__(T)
Config& push_at(const size_t& index, const T& other);

```

```

__ONLY_ALLOWED_TYPES__(T)
Config& push_at(const size_t& index, T&& other);
__ONLY_ALLOWED_TYPES__(T)
Config& push_at(const std::string& key, const T& other);
__ONLY_ALLOWED_TYPES__(T)
Config& push_at(const std::string& key, T&& other);
__ONLY_ALLOWED_TYPES__(T)
Config& push_back(const T& other);
__ONLY_ALLOWED_TYPES__(T)
Config& push_back(T&& other);
__ONLY_ALLOWED_TYPES__(T)
Config& push_back(const std::string& key, const T& other);
__ONLY_ALLOWED_TYPES__(T)
Config& push_back(const std::string& key, T&& other);

__ONLY_ALLOWED_TYPES__(T)
Config& push_after(const std::string& after_key, const std::string& key, const
T& other);
__ONLY_ALLOWED_TYPES__(T)
Config& push_after(const std::string& after_key, const std::string& key, T&&
other);

//добавить существующий список к другому списку (только при совместимости
списков)
Config& append(const Config& config);
Config& append(Config&& config);

```

## Методы удаления значений из контейнеров

(выдаст exception если тип неподходящий)

```

Config& erase_front();
Config& erase_at(const size_t& index);
Config& erase_at(const std::string& key);
Config& erase_back();

Config& erase_at(const shared_VElement::iterator iterator);

```



```

Config& erase_at(const shared_VPairElement::iterator iterator);

Config& pop_front();
Config& pop_at(const std::string& key);
Config& pop_at(const size_t& index);
Config& pop_back();

Config& pop_at(const shared_VElement::iterator iterator);
Config& pop_at(const shared_VPairElement::iterator iterator);

Config get_and_pop_front();
Config get_and_pop_at(const size_t& index);
Config get_and_pop_at(const std::string& key);
Config get_and_pop_back();

```

## Методы получения информации о параметрах объекта

```

ValueType getType() const noexcept;
bool isNull() const noexcept;
bool isBool() const noexcept;
bool isNumber() const noexcept;
bool isString() const noexcept;
bool isArray() const noexcept;
bool isJson() const noexcept;
bool isContainer() const noexcept;
bool isIndexContainer() const noexcept;
bool isMapContainer() const noexcept;

bool isEqual(const Config& other, const bool compare_comments = false, const
bool map_sort_important = false) const noexcept;
bool isEqual(const tools::IElement& other, const bool compare_comments = false,
const bool map_sort_important = false) const noexcept;
bool isEqual(const bool other) const noexcept;
bool isEqual(std::nullptr_t) const noexcept;
bool isEqual(const long double& other) const noexcept;
bool isEqual(const std::string& other) const noexcept;

```

```

//для контейнеров вернёт количество элементов
//для строк вернёт количество символов
//для null вернёт 0
//для всех остальных значений вернёт 1
size_t size() const noexcept;

bool isEmpty() const noexcept;
bool containsValue(const Config& config) const noexcept;
__ONLY_ALLOWED_TYPES_WITHOUT_CONFIG__(T)
bool containsValue(const T& other)

bool containsKey(const std::string& key) const noexcept;

```

## Операторы

```

Config& operator=(const Config& other) noexcept;
Config& operator=(Config&& other) noexcept;
Config& operator=(const tools::IElement& other) noexcept;
Config& operator=(tools::IElement&& other) noexcept;
Config& operator=(std::nullptr_t) noexcept;
Config& operator=(const bool other) noexcept;
__ONLY_NUMBER_TYPES__(T)
Config& operator=(const T& other) noexcept;
Config& operator=(long double&& other) noexcept;
__ONLY_STRING_TYPES__(T)
Config& operator=(const T& other) noexcept;
__ONLY_STRING_TYPES__(T)
Config& operator=(T&& other) noexcept;

/* WARNING: комментарии при сравнении не учитываются!
 * Учитывание комментариев только при вызове isEqual(<object>, true) */
bool operator==(const Config& other) const;
bool operator==(const tools::IElement& other) const;
bool operator==(const bool other) const;
bool operator==(std::nullptr_t) const;
__ONLY_NUMBER_TYPES__(T)
bool operator==(const T& other) const;

```

```

__ONLY_STRING_TYPES__(T)
bool operator==(const T& other) const;

bool operator!=(const Config& other) const;
bool operator!=(const tools::IElement& other) const;
bool operator!=(const bool other) const;
bool operator!=(std::nullptr_t) const;
__ONLY_NUMBER_TYPES__(T)
bool operator!=(const T& other) const;
bool operator!=(const std::string& other) const;

//числа, контейнеры(размер), строки(длина в видимых символах)
bool operator>(const Config& other) const;
bool operator>=(const Config& other) const;
bool operator<(const Config& other) const;
bool operator<=(const Config& other) const;

//контейнеры
Config& operator[](const size_t& index);
Config operator[](const size_t& index) const;
Config& operator[](const std::string& key);
Config operator[](const std::string& key) const;

Config& operator[](const std::vector<OnlySizetOrString>& complex_key);
Config operator[](const std::vector<OnlySizetOrString>& complex_key) const;
//фикс для вызова через {}
Config& operator[](const std::initializer_list<OnlySizetOrString>&
complex_key);
Config operator[](const std::initializer_list<OnlySizetOrString>& complex_key)
const;

```

## Методы для работы через итераторы

(только индексные массивы):

```

Range getRange();
const Range getRange() const;

```

```
shared_VElement::iterator array_begin();
shared_VElement::const_iterator array_cbegin() const;
shared_VElement::iterator array_end();
shared_VElement::const_iterator array_cend() const;
```

## Методы для работы через итераторы

(только массивы ключ-значение):

```
MapRange getNamedRange();
const MapRange getNamedRange() const;
shared_VPairElement::iterator map_begin();
shared_VPairElement::const_iterator map_cbegin() const;
shared_VPairElement::iterator map_end();
shared_VPairElement::const_iterator map_cend() const;
```

## Методы вывода в строку

```
//вывод без комментариев, "tabulation_level == -1" => запись в одну строку
std::string toString(const ConfigFormat format = ConfigFormat::eONLY_VALUE,
const CommentDesign &design = {}, const int8_t tabulation_level = 0) const
noexcept;
//для совместимости с STL
friend std::ostream& operator<<(std::ostream& os, const Config& config)
noexcept;
friend std::ostream& operator<<(std::ostream& os, const tools::IElement&
config) noexcept;
```

## Методы чтения файлов конфигов:

```
//return - получившийся распаршенный корневой элемент, ElementNull если не
удалось чтение
bool readFile(const std::string& file_path, const ConfigFormat format,
const CommentDesign &design = ) noexcept;
bool readFileJson(const std::string& file_path, const CommentDesign &design =
) noexcept;
```

```
bool readFileIni(const std::string& file_path, const CommentDesign &design = )  
noexcept;
```

## Методы записи файлов конфигов:

```
//return - удалось записать файл или нет  
bool writeFile(const std::string& file_path, const ConfigFormat format,  
const CommentDesign &design = {},  
const int8_t custom_tabulation_level = 0) noexcept;  
bool writeFileJson(const std::string& file_path, const CommentDesign &design =  
{},  
const int8_t custom_tabulation_level = 0) noexcept;  
bool writeFileIni(const std::string& file_path, const CommentDesign &design =  
{},  
const int8_t custom_tabulation_level = 0) noexcept;
```

## Методы для чтения конфигов из входной строки:

```
bool parse(const std::string& content, const ConfigFormat format,  
const CommentDesign &design = ) noexcept;  
bool parseJson(const std::string& content, const CommentDesign &design = )  
noexcept;  
bool parseIni(const std::string& content, const CommentDesign &design = )  
noexcept;  
bool parseYaml(const std::string& content, const CommentDesign &design = )  
noexcept;  
bool parseXml(const std::string& content, const CommentDesign &design = )  
noexcept;
```

## Статические функции чтения файлов

```
Config ReadFile(const std::string& file_path, const ConfigFormat format, const  
CommentDesign &design = {}) noexcept;  
Config ReadFileJson(const std::string& file_path, const CommentDesign &design =  
{}) noexcept;
```

```
Config ReadFileIni(const std::string& file_path, const CommentDesign &design =  
{}) noexcept;
```

## Статические функции записи файлов

```
//return - удалось записать файл или нет  
bool WriteFile(const Config& config, const std::string& file_path, const  
ConfigFormat format, const CommentDesign &design = {}, const uint8_t  
custom_tabulation_level = 0) noexcept;  
bool WriteFileJson(const Config& config, const std::string& file_path, const  
CommentDesign &design = {}, const uint8_t custom_tabulation_level = 0)  
noexcept;  
bool WriteFileIni(const Config& config, const std::string& file_path, const  
CommentDesign &design = {}, const uint8_t custom_tabulation_level = 0)  
noexcept;
```

## Статические функции чтения конфигов из входной строки

```
Config Parse(const std::string& content, const ConfigFormat format, const  
CommentDesign &design = {}) noexcept;  
Config ParseJson(const std::string& content, const CommentDesign &design = {});  
noexcept;  
Config ParseIni(const std::string& content, const CommentDesign &design = {});  
noexcept;
```

## 4.1. Формат JSON

Реализация, по большей части, соответствует стандарту RFC 8259. Чтение и запись комментариев будут рассмотрены в следующих главах.

В рамках SimpleAPI имеет тип `ValueType::eJson`.

ВАЖНО: автоматическая сортировка значений в SimpleAPI не предусмотрена.

*TODO: для ручной сортировки нужно вызвать метод `Config::sort()`.*

### Чтение конфига

Чтение организовано по принципу чтения по стандарту, с учётом поправок на условности формата:

- возможно чтение нескольких типов значений: число (long double), логический флаг (true, false, +, -), строка, пустое значение (null), массив и вложенный Json. Любое другое значение будет воспринято как строка (если не нарушает синтаксис указания значения)
- ключ с пустым значением воспринимается как null-элемент
- кавычки нужны только если строка ключа или значения содержит несколько слов
- перенос строки тоже считается разделителем, запятая необязательна
- в случае ошибки при чтении любой части документа успешно прочитанные значения не обнуляются, но заполняется описание ошибки. Метод `error()` вернёт флаг наличия ошибки. Описание доступно через вызов метода `getError()`.

### Запись конфига

Запись Json происходит строго по стандарту (если не учитывать пользовательские комментарии) для совместимости с другими парсерами при передаче по сети.

### Примеры кода

Примеры работы с Config как JSON-объектом рекомендуется изучать через написанные юнит-тесты `test_json.cpp` и `test_json_array.cpp`.



## 4.2. Формат INI

Формат INI не имеет принятого стандарта и реализуется всеми по-разному. SimpleAPI в этом плане старается брать лучшее из других реализаций. Чтение и запись комментариев более подробно будут рассмотрены в следующих главах.

В рамках SimpleAPI имеет тип `ValueType::eJson`. Полностью совместим с форматом Json, но не рекомендуется записывать в файл массив Json (`ValueType::eArray`) в формате INI, т.к. обратное чтение даст в результате иную структуру документа.

### Чтение конфига

Используется принцип "одна строка - одно значение".

Ключ может быть большой вложенности. Разделителем вложенностей ключа является слэш '/' и обратный слэш '\'.

Если переменная занимает несколько строк, то участвуют следующие правила проверки. Переменная не прочитана полностью, если выполняется хотя бы одно из правил:

- при чтении значения переменной не закрыта кавычка
- текущая строка заканчивается на знак обратного слэша '\'
- следующая строка начинается с пробела или табуляции

Каждое значение будет проверено на тип внутренним парсером. По идее, в формате INI хранятся только строки, но SimpleAPI расширяет эту идею до способности самостоятельно решать, что это за тип данных, будь то число (`long` `double`), JSON, массив в формате JSON, `bool`, пустое значение (`null`) или обычная строка (всё то же, что и в формате JSON). При этом для строк наличие кавычек необязательно.

Пробелы в начале и конце значения при обработке будут проигнорированы.

Формат INI подразумевает, что на верхнем уровне документа может существовать несколько групп. Для примера, `QSettings` от фреймворка Qt начальной группе назначает имя "MAIN". SimpleAPI для этих целей использует пустой ключ `""`. Наличие

названия группы полностью закрывает предыдущую группу. Наличие пустых строк или их отсутствие между группами не несёт логики.

Если в конфиге в рамках одной группы указано несколько одинаковых ключей, то их значения в совокупности являют собой массив с ключом в качестве имени массива.

## Запись конфига

Логика записи итогового JSON-объекта в формате INI следующая:

- *TODO: будут записаны по очереди BCE одиночные значения*
- массивы будут записаны по очереди с одним и тем же ключом
- JSON-объекты первого уровня образуют группы значений, в рамках которых все остальные внутренние контейнеры записываются как многосоставной ключ с разделителем имени через слэш '/'

## Примеры кода

Примеры работы с Config как INI-документом рекомендуется изучать через написанные юнит-тесты `test_ini.cpp` и `test_ini_array.cpp`.

## 4.3. Формат XML

TODO

## 4.4. Формат YAML

TODO

## 4.5. Комментирование значений внутри конфига

За исключением форматов XML и YAML, библиотека позволяет указать стили для чтения и записи комментариев при работе с файлами. Форматы XML и YAML имеют заданный их стандартами стиль комментариев.

### Поиск комментариев по умолчанию

По умолчанию используется стиль комментариев C/C++, а то есть `// comment` для однострочных комментариев и `/* comment */` для многострочных комментариев.

### Однострочные комментарии

Пример добавления нового шаблона поиска:

```
CommentDesign cd;  
cd.oneline_comment_variants.push_back({'#', 0});
```

Вектор принимает `std::array<char, 2>`. Оба символа являют собой строку, которая обозначает начало однострочного комментария и действует до переноса строки. Если второй символ указан как число 0 (без кавычек), то начало комментария определяется строго одним символом.

### Многострочные комментарии

Пример добавления нового шаблона поиска:

```
CommentDesign cd;  
cd.multiline_comment_variants.push_back({'{', '|', '}'});
```

Вектор принимает `std::array<char, 3>`. Суть описания похожа на однострочные комментарии за тем исключением, что наличие третьего символа обозначает замыкающую границу комментария.

Примеры использования:

- `{'{' , '|' , '}'}` -> `{| multiline comment |}`
- `{'{' , 0 , '}'}` -> `{ multiline comment }`
- `{'{' , '|' , 0}` -> `{| multiline comment |{`

## Важные замечания

1. Комментарии не будут прочитаны или записаны, если поле `with_comments` в составе объекта `CommentDesign` не равно `true`.
2. Переменная объекта `CommentDesign` в составе `Config` будет обновлена и дополнена при чтении нового файла/входной строки конфига.
3. Для многострочных комментариев можно ограничить ширину колонки. Строки будут обрезаны автоматически. Для этого следует использовать поле `opt_multiline_column_size`. Значение 0 отключает параметр.
4. Для многострочных комментариев можно указать стиль рамки, например:

```
CommentDesign cd;
cd.opt_multiline_border = '*';
```

Выведет так:

```
/******
 * comment *
*****/
```

Значение 0 отключает параметр.

5. Для многострочных комментариев без рамки вывод по умолчанию указывает начало и конец комментария на отдельных строках, например:

```
/*
multiline
```

```
comment
*/
```

Поле `opt_multiline_border_at_content_line=true` изменяет этот момент:

```
/* multiline
    comment */
```

6. Комментарий можно указать ДО значения и ПОСЛЕ него. Несколько комментариев ДО значения сформируют один большой комментарий. Комментарий ПОСЛЕ значения применяется только в том случае, если после написания значения комментарий был начат до переноса строки. Комментарий ПОСЛЕ значения может быть только одним:

```
{
    // много
    // комментариев
    // ДО значения
    key="value", // комментарий ПОСЛЕ значения
    /* ещё
    один пример */
    /* вторая часть второго примера */
    key2=12465 /* большой
        комментарий
        после значения
        */
}
```

7. При записи комментария ПОСЛЕ значения параметр `opt_multiline_column_size` не влияет на ширину колонки, если в комментарии нет хотя бы одного явного переноса строки (`\n`). В таком случае будет сделана попытка записать комментарий как однострочный (для лучшей читабельности итогового конфига).

## **5. Сетевое взаимодействие**



## 5.1. Сокеты

## **5.2. Поток для автоматизации работы сокетов**

## 5.3. Сообщения (класс Message)

## **5.4. Функции-утилиты для сетевого взаимодействия**

## **6. Дополнительные функции- утилиты**